GPU Acceleration of the ORB-SLAM3 Front End

Engineering note for Chapter 3 (implemented system), with its validation measurements

Bolton Athit Davies

Reporting cutoff: 29 October 2026; revised 3 November 2026

Purpose of this document. This note records what was changed in the ORB-SLAM3 fork to move its feature front end onto the GPU, how the change was validated, and what the validation runs revealed. It is written for Chapter 3 (the implemented system and its planned-versus-implemented table) and for the threats-to-validity discussion of Chapter 4. It is not a result slot: no Chapter 4 result uses this code yet, and the simulation configurations are unchanged. Every number traces to a file under `output/compare/real/` named in section 7.

Status: completed. Implemented and validated on the laptop (RTX 3070) for the real-rig configuration: extractor patch (10) and stereo-matcher patch (10b), enabled by default in `orbslam3_ros2/config/wil/stereo*.yaml`; a pre-existing relocalisation crash found during validation is fixed (patch 11), and the validation of this and the following days found five more out-of-bounds reads, an Eigen temporary read after its scope and a layout mismatch between node and library, all fixed (patches 12 to 14b, node build flag); an AddressSanitizer run over a whole real recording is clean. The stereo-inertial mode, which shares this front end, reached a reproducible 0.48 m on `dataset_realv6_000` on 3 November (section 5). Portability to the Jetson platform is **untested**. The fork changes are not yet committed.

1 Why the front end was moved

On the real rig the tracker could not keep up with the camera: it took about 85 ms per stereo pair against a 33 ms frame period, so two frames in three were dropped. Roughly 80 % of that time was ORB extraction [1], one CPU thread per camera computing FAST [2] on an eight-level pyramid of a 1920×1080 fisheye image, then the rotated BRIEF descriptor [3]. Dropped frames are not a uniform loss: they cluster in turns, where inter-frame motion is largest, which is where the real-rig trajectories were observed to fail.

The remaining cost after extraction was the stereo matcher. ORB-SLAM3 [4] matches the two fisheye views by brute force over every descriptor pair inside the lapping area, which on this rig is the full image width, so about 2000×2000 Hamming distances are computed per frame. Table G.1 gives the stage split measured in isolation on two recorded frame pairs.

2 What moved and what did not

The per-pixel and per-keypoint arithmetic moved; the selection logic stayed. On the GPU: the FAST-9 corner score (a literal port of OpenCV's `cornerScore<16>`), non-maximum suppression with ORB-SLAM3's two-threshold cell rule, the 7×7 Gaussian blur, the intensity-centroid orientation and the rotated BRIEF descriptor; since patch (10b) also the brute-force Hamming 2-nearest-neighbour stereo matching. On the CPU, unchanged: the image pyramid, the dynamic-object mask veto used by the Experiment 3 filter, the octree keypoint distribution that gives ORB-SLAM3 its uniform coverage, and everything downstream of the frame.

Table G.1: Time per stereo frame of the fisheye stereo front end, measured in a standalone harness on two `dataset_realv5_000` frame pairs (1920×1080, 2000 features per camera, 8 levels). Mean of 10 repetitions after a warm-up. The CPU matcher figure is with all 16 hardware threads idle; single-threaded it is 38.2 ms.

Stage	CPU (ms)	GPU extractor (ms)	GPU extractor + matcher (ms)
Extraction, both cameras (two threads)	41.8	12.1	12.1
Brute-force Hamming 2-NN matching	9.2	8.2	0.44
Ratio test	0.1	0.1	0.1
Triangulation of accepted matches	0.2	0.2	0.2
Sum	51.3	20.6	12.8

Keeping the selection on the CPU is what makes the output comparable. The GPU produces the same candidate set and the same scores, and the stock distribution picks from them, so a frame extracted on the GPU is the frame ORB-SLAM3 would have extracted, up to tie-breaks between corners of equal score (section 3.1). The fisheye camera model [5] and the pinhole stereo path are untouched, so the simulation configurations, which keep the switch off, run the stock code.

No CUDA toolkit is needed to build the fork. The driver library and the NVRTC runtime compiler [6] are loaded with `dlopen` on first use and the kernels are compiled from a source string at start-up, once per process (about 90 ms). If any step fails, the extractor logs one line and runs the CPU path. Table G.2 lists the three settings that control the path.

Table G.2: Settings that control the GPU front end. Both real-rig configurations set `ORBextractor.useGPU: 1`; both simulation configurations omit it.

Setting	Meaning
`ORBextractor.useGPU` (YAML, 0/1)	GPU extraction; absent = 0.
`Stereo.useGPUMatch` (YAML, 0/1)	GPU stereo matching; absent = follow `useGPU`. Used only for A/B runs.
`ORB_SLAM3_NVRTC_DIR` (environment)	Directory of `libnvrtc.so.12` when it is not on the loader path.

3 Validation

3.1 Fidelity of the GPU extractor

The GPU extractor reproduces the CPU extractor to within tie-breaks. Table G.3 compares the two on four recorded frames: nine in ten GPU keypoints sit at the identical pixel and octave of a CPU keypoint, and on those the orientation is equal and the descriptors differ by a third of a bit on average. The remaining tenth are corners of equal FAST score that the GPU's suppression order and the octree resolve differently; they are not a different detector.

3.2 Fidelity of the GPU stereo matcher

The GPU matcher is exact. On the two frame pairs of table G.1 it returned, for every one of the 2003 and 2005 query descriptors, the same best index and the same two distances as OpenCV's brute-force matcher, including its tie order, so the ratio test downstream accepted the identical sets (391 and 387 matches). Its cost is 0.44 ms per frame against 8.2–10.0 ms.

Table G.3: CPU versus GPU extractor on four `dataset_realv5_000` frames (1920×1080, 2000 features, 8 levels, RTX 3070). "Identical" means the same pixel position and octave; the descriptor comparison is over those keypoints. Time is the mean of 15 repetitions.

Quantity	Value
Time per image, CPU	39 ms
Time per image, GPU	9.3 ms
Keypoints per level, difference	±2
GPU keypoints identical to a CPU keypoint	90 %
Orientation difference on identical keypoints	0.00°
Descriptor Hamming distance on identical keypoints, mean	0.35 bits
Descriptors bit-identical / within 2 bits	73 % / 99 %

3.3 Effect inside the running node

In the node the gain is smaller than the harness suggests, because the stages that remain on the CPU become the larger share. Table G.4 reports the controlled comparison for the stereo matcher: the tracker falls from 64 ms to 45 ms per frame at real-time replay, 35 % more frames are processed, and the inlier and map-point counts are unchanged. The two runs of each arm agree to within 1 ms. Figure G.1 shows where the remaining 45 ms go; it is still above the frame period, so frames still drop at 30 Hz, fewer than before.

Table G.4: Node A/B of the stereo matcher on `dataset_realv5_000`, first 60 s of the recording, stereo-only, loop closing off, viewer off, ABBA order, each run started with the CPU package below 75 °C. Both arms use the GPU extractor. Medians over frames in the tracking-OK state. Source: `output/compare/real/gpu-stereo-matcher-ab/gpu_match_abba_metrics.csv`.

Rate	Arm	Frames processed	Dropped (%)	Extraction+stereo (ms)	Tracker total (ms)	Inliers (median)	Map points (final)	Losses
1.0	A1 CPU matcher	840	47	27.3	64.2	831	2498	0
1.0	B1 GPU matcher	1153	34	14.8	44.7	851	2500	0
1.0	B2 GPU matcher	1147	35	15.1	44.5	844	2444	0
1.0	A2 CPU matcher	866	45	27.0	63.2	820	2711	0
0.5	A1 CPU matcher	1487	12	26.1	64.9	878	2734	0
0.5	B2 GPU matcher	1660	7	14.4	50.1	917	2854	0
0.5	A2 CPU matcher (started at 79 °C)	1088	29	31.8	84.0	853	2883	0
0.5	B1 GPU matcher (started at 95 °C)	89	–	–	–	–	–	crashed, section 4

The extractor itself was compared in the node a day earlier without thermal control (full recording, CPU arm then GPU arm back to back): tracker 110–82 ms at half speed and 206–118 ms at real time, frames processed 23 % and 64 %, inliers and map points at parity, no tracking loss in any run. The GPU arm ran second and still won, so the direction is safe; the magnitudes are not controlled and are not quoted further.

3.4 A measurement condition found on the way

Back-to-back timing runs on this laptop are confounded by heat. In the first matcher A/B, every run slowed from about 60 to 150 ms per frame as the CPU package passed 90 °C and throttled, so whichever arm ran second looked worse regardless of code (`order_confounded/` in the same directory). The ABBA design with cool starts above was adopted in response, and the same rule applies to every capacity table of Section 6.4 of the evaluation protocol: start below 75 °C, alternate arms, keep windows short, and do not read effects under about 20 % from full-recording frame counts.

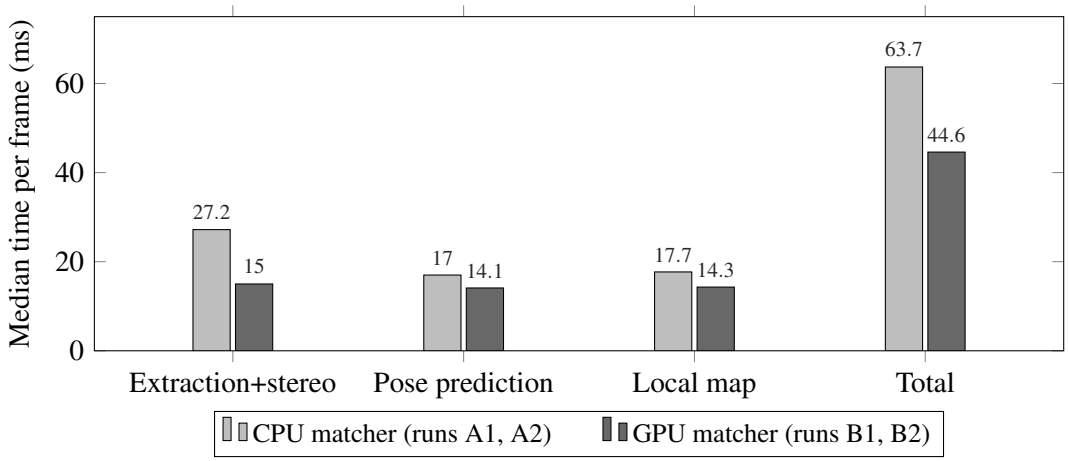


Figure G.1: Tracker time per frame by stage, mean of the two runs per arm at replay rate 1.0 from table G.4 ($n = 2$ runs per arm, 840–1153 frames per run). The two CPU-side stages also fall by about 3 ms: the CPU matcher launched a sixteen-thread parallel loop on every frame that competed with the other threads. Data: `gpu_plugin_stage_means.csv` beside this note.

4 A defect exposed by the validation runs

Two validation runs ended in the segmentation fault that had been killing real-rig runs after a tracking loss since the realv4 recordings ("exit code −11"). Caught under a debugger, the fault is in the relocalisation matcher of ORB-SLAM3, not in the GPU code. For a fisheye camera the distortion vector handed to the frame is zero, so the "undistorted" keypoint list is a copy of the left-camera keypoints only, while the keyframe's map-point list spans both cameras; a map point first observed through the right camera indexes past the end of that list, the resulting angle is not a number, and the orientation histogram is indexed with `INT_MIN`. Every other matcher routine on the tracking path already guards this index; the relocalisation one did not, which is why the crash appeared only on the fisheye rig and only after a loss.

The fix (patch 11) applies the same guard and keeps the original line as a comment. On the patched library, forced relocalisations (a map built from the first 25 s, then playback restarted at three other offsets, twice, under the debugger) completed without a fault where the unpatched library had crashed in four of the six runs that reached that path. Backtrace, registers and the verification logs are under `output/compare/real/relocalisation-crash-patch11/`. For the report this means every earlier real-rig run that lost tracking was cut short by this defect rather than by the estimator's own recovery behaviour, and the real-robot slots recorded after 29 October 2026 are the first on which a tracking loss can be followed through.

5 Stereo-inertial on the same recording

The inertial mode shares the GPU front end, and until this afternoon it did not hold tracking on `dataset_realv5_000` at all (ten map resets in thirty seconds). The cause was not the front end. Three defects were found in order, each by measurement, and table G.5 gives the run after each fix.

Accelerometer scale. The launch file multiplied acceleration by 1.0589, a correction measured on the previous (RealSense) IMU, which read 9.26 m s^{-2} at rest. The rig's current IMU reads 9.886 m s^{-2} at rest (stillest 2 s window of the recording, standard deviation 0.018), so ORB-SLAM3 was fed 10.47 m s^{-2} against its fixed 9.81 and its inertial initialisation never converged. With the measured factor, 0.9923, initialisation completed at 3.2 s.

IMU delivery. The IMU then arrived in bursts: zero messages between most frames, then up to 1063 at once, while the image stream flowed one frame at a time, so half of the frames were skipped for want

Table G.5: Stereo-inertial runs on `dataset_realv5_000` at replay rate 0.5, GPU front end on, loop closing on, one run per row. "Starved" frames are frames skipped because fewer than two IMU samples had arrived for them. Source: `output/output_orb/real/realv5_imu_*/run_summary.csv`.

Configuration	IMU received	Starved	Resets	Loss events	Map points	Poses
scale 1.0589 (old IMU), default buffers, first 30 s	6370	146	10	30	–	202
scale 0.9923, default buffers	22409 (83 %)	1063	19	73	3140	1060
scale 0.9923, 64 MB socket buffers	26813 (99.8 %)	615	19	78	177	1657
+ IMU in its own callback group (reliable)	26838	**2**	**2**	11	**9768**	**2318**
+ IMU in its own group, best-effort	26854	0	1	1	10068	1632

of IMU data and the map reset on every timestamp jump. Two layers were separated by two changes. Raising the kernel socket buffers from the Linux default of 208 KiB to 64 Mi B stopped the loss (83 % to 99.8 % of messages delivered) but not the bursts. Giving the IMU subscription its own reentrant callback group, so that it never queues behind a JPEG decode in the node's default mutually exclusive group, removed the bursts: starved frames 615 to 2, resets 19 to 2. A best-effort subscription was added as a second lever and was not better (no starvation, but more dropped frames and a slower tracker); the default stays reliable.

What was left on 29 October, and what became of it. The two remaining resets at 3.0 and 5.6 s are ORB-SLAM3's own IMU check during initialisation; they appear in every inertial run since and are benign. The lever arm was measured by tape that evening (left camera 0.26 m above and 0.065 m in front of the IMU) and entered under the rotation already in the file; the sign of the IMU's z axis and the lateral offset remain unverified. On `dataset_realv6_000` the inertial mode then proved unstable in a way none of this explained: identical runs scored between 0.4 and 9.6 m. The resolution, documented in the engineering record (sections 7.10 to 7.13) and in the Experiment 3 note, was that the optimiser trusted an IMU whose axis signs, lever arm and time offset nobody had verified; raising the configured IMU noise densities tenfold gave 0.49, 0.47 and 0.49 m in three consecutive runs at real-time replay, with the GPU front end on. Those three runs, not the inertial rows of table G.5, are the state of the inertial mode.

Resolution of these runs is not recorded in their manifests: the real-rig inertial configuration carried `Camera.newWidth/newHeight` = 960/540 for part of the afternoon (since commented out), so the inertial rows of table G.5 may be at half resolution and are not comparable with the stereo-only timings above.

6 Limitations and threats to validity

- **Platform.** Validated on the development laptop (RTX 3070, driver 595) only. The real robot's Jetson Xavier NX has not been tried; its driver ships the same two libraries, but neither the runtime compilation nor the timings have been confirmed there.
- **Reproducibility.** GPU extraction is not bit-reproducible run to run, so repeated real-rig runs are statistical, as the simulation runs already are; the matcher is exact.
- **Shared GPU.** In Experiment 3 the YOLO detector uses the same GPU. Contention between the detector and the front end was not measured.
- **One recording.** All node measurements are on `dataset_realv5_000`. The GPU timings are stereo-only; the stereo-inertial runs of section 5 are counts of resets and starved frames, not timings, and their resolution is uncertain as flagged there.
- **Configuration generation.** The real-rig configuration now differs from every earlier real recording's run (GPU front end, 28 October extrinsics, re-measured accelerometer scale, IMU callback group, measured lever arm). Earlier real-rig ORB-SLAM3 results are not repeats of it and are not pooled with it.
- **Not yet committed.** The fork carries the patches of 29 October to 3 November uncommitted, the ROS package its node, build-flag and configuration changes; run manifests cannot cite a commit for

any of it until they are committed.

7 Reporting rules

Every number in this note traces to one of the following files.

- Harness measurements (tables G.1 and G.3): harness sources and the four frame images beside the A/B artefacts, `output/compare/real/gpu-extractor-ab/gpu_orb_test.cc` and `output/compare/real/gpu-stereo-matcher-ab/stereo_stage_test.cc`.
- Node measurements (table G.4 and figure G.1): per-run `tracking_frontend.csv` and `run_summary.csv`, the metrics table and the start/end temperatures, all under `output/compare/real/gpu-stereo-matcher-ab/`.
- Crash evidence (section 4): backtraces, registers and the verification logs under `output/compare/real/relocalisation-crash-patch11/`.
- Stereo-inertial runs (table G.5): `run_summary.csv` and `performance.csv` under `output/output_orb/real/realv5_imu_*/` and, for the first row, `~/output/wil_orbslam3_stereo_imu/`.
- IMU rest value and arrival statistics: the bag itself and `script/imu_arrival_probe.py`.
- Engineering record: `orbslam3_ros2/MEMORY_CLAUDE/integration.md`, Sections 3.5, 3.5.1, 7.10 and 7.11.

References

[1] Rublee, E., Rabaud, V., Konolige, K. and Bradski, G., 2011, "ORB: An Efficient Alternative to SIFT or SURF," **Proceedings of the IEEE International Conference on Computer Vision (ICCV 2011)**, 6–13 November 2011, Barcelona, Spain, pp. 2564–2571.

[2] Rosten, E. and Drummond, T., 2006, "Machine Learning for High-Speed Corner Detection," **Computer Vision – ECCV 2006, Lecture Notes in Computer Science, Vol. 3951**, 7–13 May 2006, Graz, Austria, pp. 430–443.

[3] Calonder, M., Lepetit, V., Strecha, C. and Fua, P., 2010, "BRIEF: Binary Robust Independent Elementary Features," **Computer Vision – ECCV 2010, Lecture Notes in Computer Science, Vol. 6314**, 5–11 September 2010, Heraklion, Greece, pp. 778–792.

[4] Campos, C., Elvira, R., Gómez Rodríguez, J.J., Montiel, J.M.M. and Tardós, J.D., 2021, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multi-Map SLAM," **IEEE Transactions on Robotics**, Vol. 37, No. 6, pp. 1874–1890.

[5] Kannala, J. and Brandt, S.S., 2006, "A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses," **IEEE Transactions on Pattern Analysis and Machine Intelligence**, Vol. 28, No. 8, pp. 1335–1340.

[6] NVIDIA Corporation, 2026, **NVRTC: CUDA Runtime Compilation User Guide** [Online], Available : `https://docs.nvidia.com/cuda/nvrtc/index.html` [2026, October 29].